

# Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Export**

**Apetrei Diana-Andreea**

Grupa: 30231

Mai 2026

# Cuprins

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>Introducere și Contextul Sistemului</b>                | <b>2</b> |
| 1.1      | Harta de Context (Context Map) . . . . .                  | 2        |
| <b>2</b> | <b>Arhitectura Internă a Microserviciului Export</b>      | <b>2</b> |
| 2.1      | Diagrama de Pachete (Arhitectura Hexagonală) . . . . .    | 2        |
| 2.2      | Diagrama de Clase Detaliată . . . . .                     | 3        |
| <b>3</b> | <b>Design-ul Bazei de Date (Stateless)</b>                | <b>4</b> |
| <b>4</b> | <b>Modelarea Domeniului (Domain-Driven Design)</b>        | <b>4</b> |
| 4.1      | Aggregate Root — <code>FisierExportat</code> . . . . .    | 5        |
| 4.2      | Domain Services — <code>IExportStrategy</code> . . . . .  | 5        |
| <b>5</b> | <b>Șabloane de Proiectare (Design Patterns) Utilizate</b> | <b>6</b> |
| 5.1      | Șablonul Strategie (Strategy Pattern) . . . . .           | 6        |
| 5.2      | Șablonul Singleton (Gestionat prin IoC) . . . . .         | 6        |
| <b>6</b> | <b>Concluzii</b>                                          | <b>6</b> |

# 1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

## 1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale, fiecare acționând ca un microserviciu independent: *Zboruri*, *Bilete*, *Utilizatori*, *Notificări*, *Statistici* și *Export*.

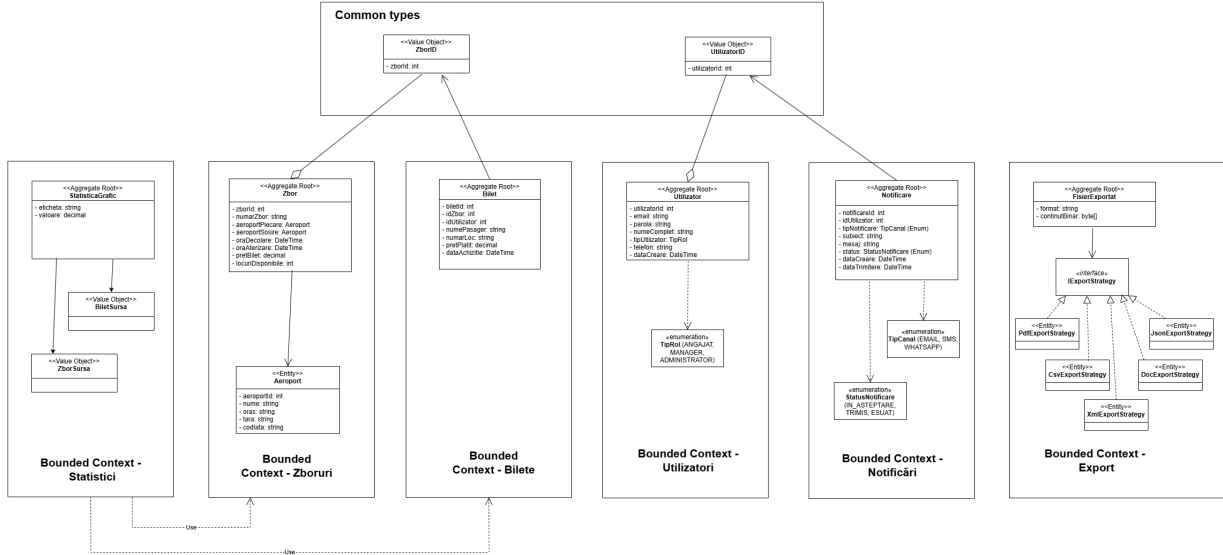


Figura 1: Harta de context a sistemului global de gestiune

**Justificarea deciziei:** Microserviciul **Export** a fost conceput ca un modul complet *stateless* (fără stare) și decuplat. Spre deosebire de modulul *Zboruri* care gestionează persistența propriilor entități, *Export* acționează ca un serviciu pur utilitar. El primește date brute sau agregate (de exemplu de la *Statistici* sau *Zboruri*) și le transformă în formate portabile (PDF, DOC, CSV). Această separare arhitecturală permite microserviciului de export să fie scalat independent de restul sistemului în momentele de vârf, atunci când cererea pentru generarea de rapoarte este intensă.

## 2 Arhitectura Internă a Microserviciului Export

### 2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Export** respectă principiul Inversării Dependențelor (*Dependency Inversion Principle*) din Arhitectura Hexagonală, menținând nucleul de business izolat de framework-uri.

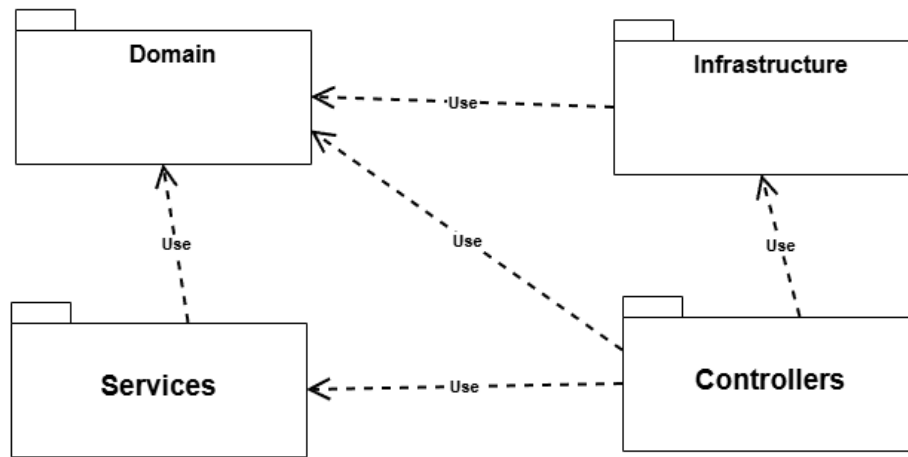


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

**Justificarea deciziei:** Direcția dependențelor (marcată prin *-Use-*) indică orientarea clară către interior, spre pachetul **Domain**. Domeniul expune contractul (**IExportStrategy**), dar nu este conștient de instrumentele externe folosite. Stratul de **Infrastructure** depinde de domeniu, implementând strategiile tehnologice concrete (folosind librării precum Jackson, Apache POI sau iTextPDF). Pachetul **Controllers** se ocupă de preluarea cererilor HTTP de la client, iar **Services** orchestrează fluxul. Această compartimentare garantează că schimbarea unei tehnologii de generare de fișiere nu va afecta endpoint-urile.

## 2.2 Diagrama de Clase Detaliată

Structura granulară a claselor detaliază asocierile și realizările necesare funcționării microserviciului de export.

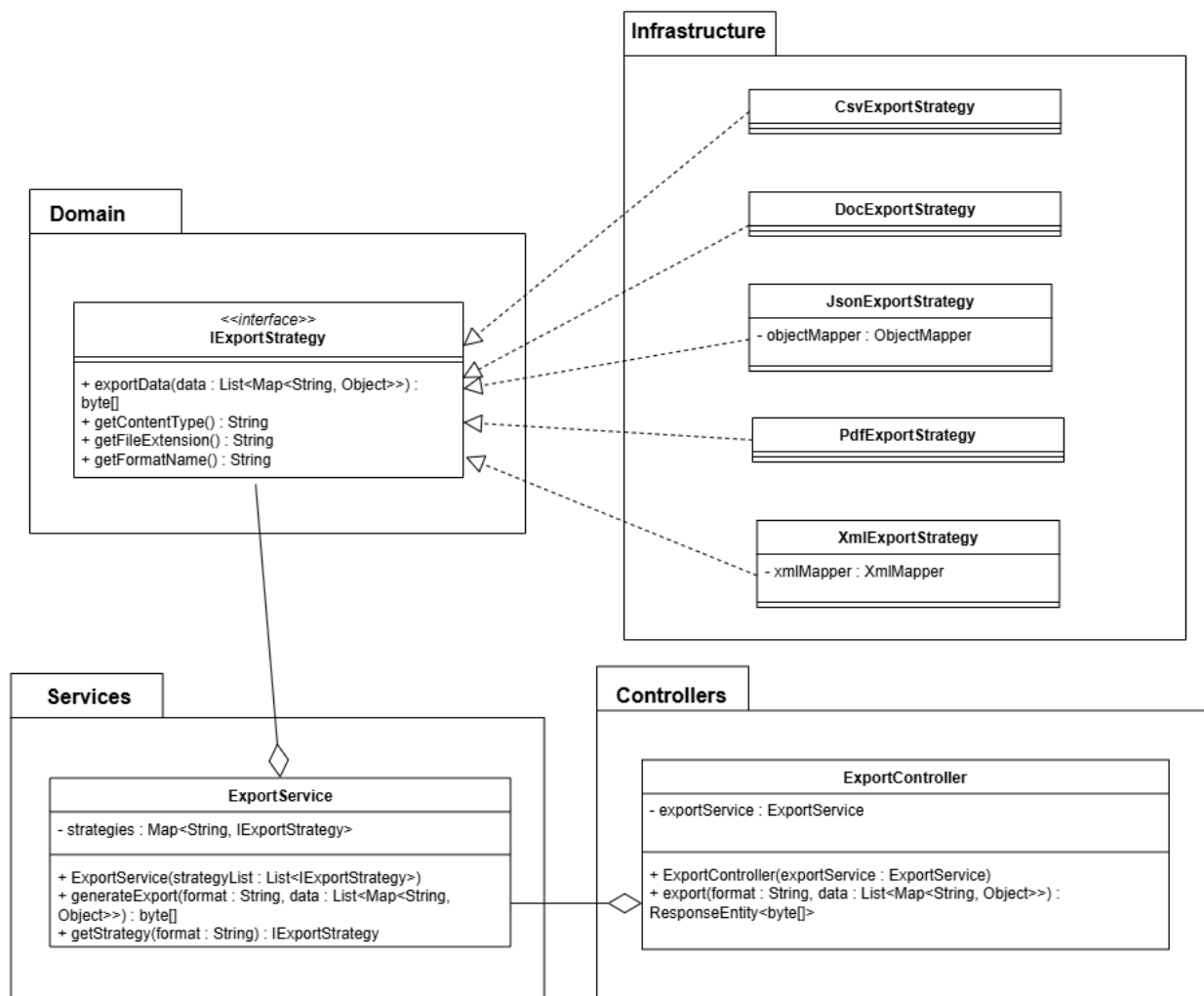


Figura 3: Diagrama de clase detaliată a microserviciului Export

**Justificarea deciziei:** Așa cum reiese din figura 3, Controller-ul (`ExportController`) delegă prelucrarea datelor către `ExportService`. Flexibilitatea reală provine din asocierea dintre Serviciu și Domeniu: `ExportService` deține o colecție abstractă (`Map`) formată exclusiv din referințe către interfața `IExportStrategy`. Extensiile concrete (`PdfExportStrategy`, `CsvExportStrategy`, etc.) nu sunt strâns cuplate la serviciu, ci decizia se ia la momentul rulării (runtime) pe baza formatului solicitat.

### 3 Design-ul Bazei de Date (Stateless)

Spre deosebire de modulele *Zboruri* sau *Utilizatori*, microserviciul **Export** este guvernat de un principiu arhitectural distinct: **nu posedă o bază de date proprie**.

**Justificarea deciziei:** Întregul flux funcționează ca o operațiune tranzitorie de procesare *in-memory*. Aplicația client expediază datele printr-un payload, iar microserviciul răspunde direct cu matricea de octeți (`byte[]`) a fișierului rezultat. Stocarea acestor documente local sau într-o bază de date asociată serviciului ar fi inutilă și ar genera date redundante, încălcând principiul "Single Source of Truth". Lipsa persistenței face modulul rapid, rezilient și extrem de ușor de întreținut.

### 4 Modelarea Domeniului (Domain-Driven Design)

Deși acest microserviciu are o structură a stării redusă, el aplică o modelare orientată pe domeniu adecvată operațiunilor sale.

## 4.1 Aggregate Root — `FisierExportat`

Analizând harta de context, domeniul de export este definit conceptual de agregatul `FisierExportat`. Acesta reprezintă entitatea finală rezultată, descrisă de:

- **Formatul** (`format: string`): Tipul fișierului, care alege strategia aplicabilă.
- **Conținutul Binar** (`continutBinar: byte[]`): Fluxul efectiv de date rezultat în urma prelucrării și mapării.

## 4.2 Domain Services — `IExportStrategy`

Axa centrală a logicii de domeniu o constituie interfața `IExportStrategy`, acționând ca un "Domain Service". Aceasta standardizează contractual modul în care orice tip de document va fi produs și furnizat sistemului, incluzând expunerea metadatelor (tipul MIME și extensia de descărcare).

## 5 Șabloane de Proiectare (Design Patterns) Utilizate

### 5.1 Șablonul Strategie (Strategy Pattern)

Pentru a elimina logica greoaie decizională (scutind aplicația de blocuri multiple `if-else` sau `switch` privind formatul), microserviciul bazează pe pattern-ul comportamental **Strategy**.

**Explicație și Justificare:** Contractul `IExportStrategy` reprezintă interfața tiparului. Clasele din infrastructură (`JsonExportStrategy`, `DocExportStrategy` etc.) sunt strategiile concrete. `ExportService` doar primește implementarea potrivită de la Spring și apelează o metodă unică. Arhitectura sprijină 100% principiul Open-Closed: dacă în viitor se dorește adăugarea formatului Excel (`.xlsx`), se va introduce doar o nouă clasă-strategie, fără modificări în Controller sau Service.

### 5.2 Șablonul Singleton (Gestionat prin IoC)

Performanța sistemului este ajutată masiv de pattern-ul **Singleton**, implementat implicit prin funcționalitatea de Inversion of Control (IoC) a framework-ului Spring.

**Explicație și Justificare:** Serviciile (`@Service`) și strategiile de generare (`@Component`) sunt instanțiate o singură dată la inițializarea containerului Spring. Deoarece nicio strategie nu memorează vreo stare client în atributele sale (cu excepția obiectelor utilitare sigure precum `ObjectMapper`), funcțiile lor pot fi apelate concurrent de mii de utilizatori simultan (thread-safe execution). Asta elimină overhead-ul de a crea noi obiecte complexe pentru fiecare cerere de raport primit.

## 6 Concluzii

Microserviciul **Export** ilustrează un design robust centrat exclusiv pe procesarea datelor în memorie. Prin renunțarea la persistență și implementarea cuplajului slab mediat de *Strategy Pattern* și Arhitectura Hexagonală, rezultă o componentă rapidă, scalabilă, facil de testat și pregătită oricând pentru integrarea nativă a unor noi tipuri de conversie. Predictibilitatea operațiunilor rămâne calitatea supremă a acestei abordări.